

PlantProcess IQ -- Platform Capabilities

Document class	Capability Reference
Audience	Evaluators, engineers, administrators
Product	PlantProcess IQ (PPIQ) -- SOU Industrial Software
Version	1.0 -- June 2026

1. Generic source integration

PlantProcess IQ is designed to connect to the common source types found in a plant landscape -- Microsoft SQL Server, PostgreSQL, MySQL, Oracle, and file-based sources such as Excel and CSV -- without any change to the product. An administrator links a source through the configuration interface, selects a provider, and the system tests connectivity before the link is saved. Credentials are masked on read-back. The administrator then selects which tables to import and sets a read cadence per object, ranging from a few minutes to several days. The connector catalog declares its certified availability honestly per source and version rather than overstating readiness.

2. Staging layer

Each imported object is held in a staging area in its original, source-shaped form -- not yet in the product's internal structure. On every scan the system compares the last index already held against the last index in the source and imports only new records. Each run writes an import batch recording rows, duration, the watermark reached, and any errors. Every import is a named job with a defined cycle and a visible status.

3. Mapping and genealogy engine

This is the heart of the product. PlantProcess IQ turns the many foreign keys of many systems into one canonical fact, generically. Joins start from business keys and are strengthened through normalization, mapping views, genealogy, and a confidence score. A business-key dictionary reconciles mismatched identifiers explicitly -- for example, an identifier in one system equated to a differently formatted identifier in another -- and rejects conflicts rather than silently merging them. Mapping is authored as safe, read-only SQL views in which write and schema-changing statements are rejected, an implicit row limit and statement timeout apply, and the plan is checked before publication. The result is a genealogy that can be walked in both directions, from a finished-product defect back to upstream process and chemistry, and forward again, on the customer's own key names.

4. KPIs, dashboards, and widget customization

KPIs are first-class, versioned SQL views, consumed both by dashboard widgets and by the learning jobs. Non-developers build pages by selecting widgets from a library and binding them to data without writing an endpoint, and a script layer allows transforms such as grouping a chart by shift. Tables virtualize for

large datasets, charts respond to filtering and sorting, and a long-running load always shows progress rather than hanging.

5. Analysis and learning jobs

PlantProcess IQ runs analytical jobs on demand and on a schedule -- relating process parameters to defects, to downtime, and to KPI outcomes. Results are produced by deterministic engines that run inside the tenant. Every result states its method, sample size, filters, excluded records, and stability, and is framed as a suspected contributor. The methodology is described in the dedicated methodology document.

6. Quantified value

A value engine converts a finding into a bounded estimate -- a range with an explicit abstain path when the data does not support a number -- computed on the plant's own data, with every input drillable to its source. This lets a finding be weighed in economic terms without overstating precision.

7. Jobs monitor and operations

Every job -- import, correlation, learning, maintenance -- appears in a jobs monitor showing its last run, outcome, duration, and impact on the source. Schema changes in a source raise a typed change event; dependent mappings are flagged and dependent imports are paused rather than allowed to produce wrong facts, and a mapping-health panel reports green, degraded, or broken status with the reason.

8. Roles and editions

Access is scoped by role, so an executive, an engineer, an administrator, and an operator each see the pages and edit rights appropriate to them. Feature availability is governed by edition, and entitlements are resolved by a single backend authority consulted by both the interface and the API, so a capability cannot be reached by calling the API directly. Editions and their scope are summarized in the licensing material provided with a proposal.